Object-Oriented Programming

Abstract Class and Interface

Ridwan Rismanto

What will we learn

- Concept of abstract class
- Abstract method
- UML notation for abstract class
- Concept of interface
- Difference between interface and abstract class
- How to create a class that implements an interface
- UML notation for interface and implements relationship

Abstract Class

- Describes something that is general in nature, which can only function after it is described in a more specific form.

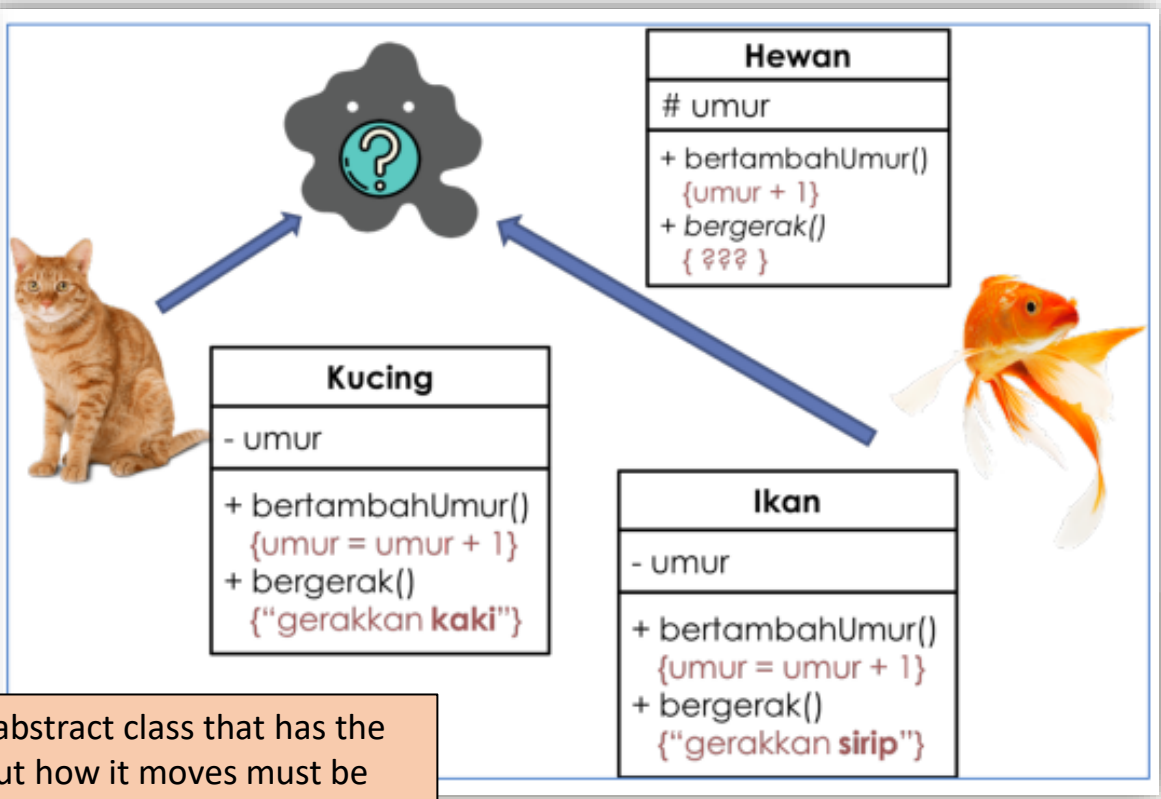
- **Animal**
  - Can all animals age?
    - YES
  - Can all animals move?
    - YES
  - How do animals move?
    - Depending on what kind of animal it is..

Cat

- Is a cat an animal?
  - **YES**
- Because a cat is an animal, it means a cat can age and move, right?
  - YES
- How does a cat move?
  - By stepping with its legs

Fish

- How does a fish move?
  - With its fins



Therefore, Animals are an abstract class that has the behavior of movement, but how it moves must be detailed in each of its subclass.

Why must there be an abstract class?

- To define standard methods that must exist in every subclass, and those standard methods must be redefined (or recoded) within their subclasses.
- Thus, by creating an abstract class, we can establish a kind of 'rule' for its subclasses.
- Why must we create rules?
  - So we can ensure that the subclasses, when called later, have their standard methods guaranteed to exist and can be called in the same way (same parameters and same return value).
  - For example, the previously mentioned animal. All subclasses that derive from the animal class must have a move() method.
    - However, how each of those animals moves must be redefined (re-coded) within their respective class.

Abstract Class

- An Abstract Class is a Class that cannot be instantiated but can be extended.
- An abstract class can only be utilized when it is extended.
- How to create an abstract class:

```
3    abstract class Phone
4    {
5        protected String phoneNumber;
6    }
```

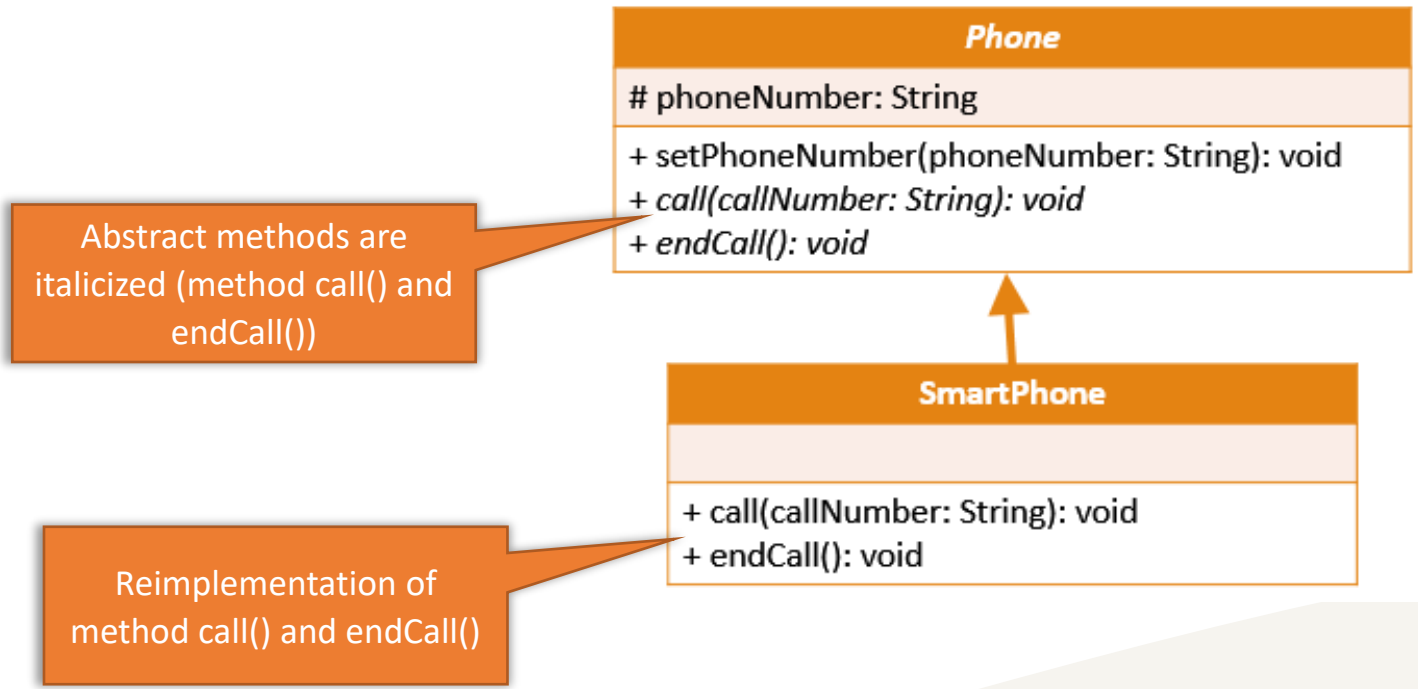
Abstract Method

- *An abstract method is a method that is only declared and has no implementation (ended with a semicolon at the end of the declaration, and without curly brackets).*
- Example of an abstract method:

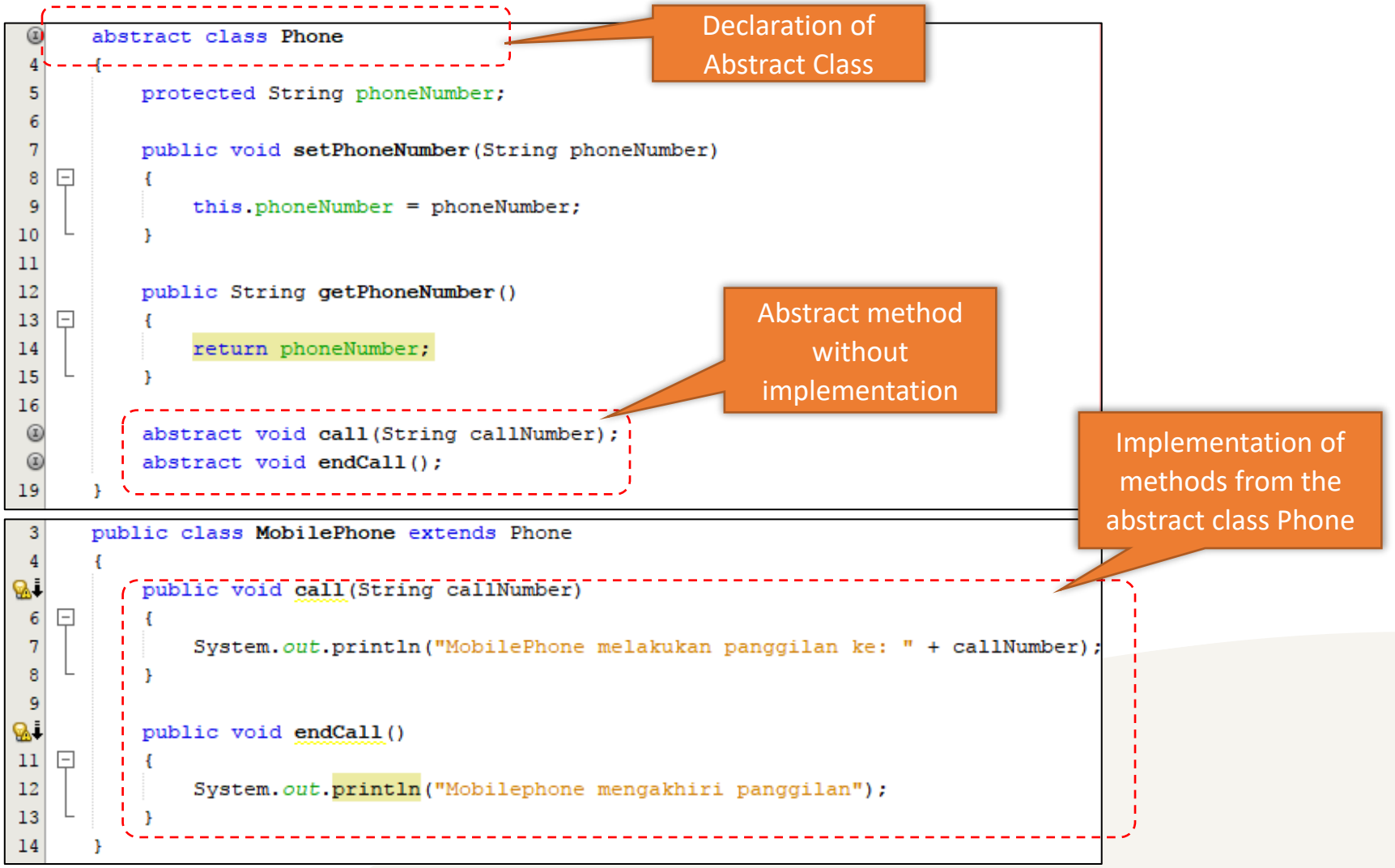
```
3    abstract class Phone
4    {
5        protected String phoneNumber;
6
7        abstract void call(String callNumber);
8        abstract void endCall();
9    }
```

Inheritance in an Abstract Class

- When a class inherits an abstract class, it must reimplement all the abstract methods present in its parent.



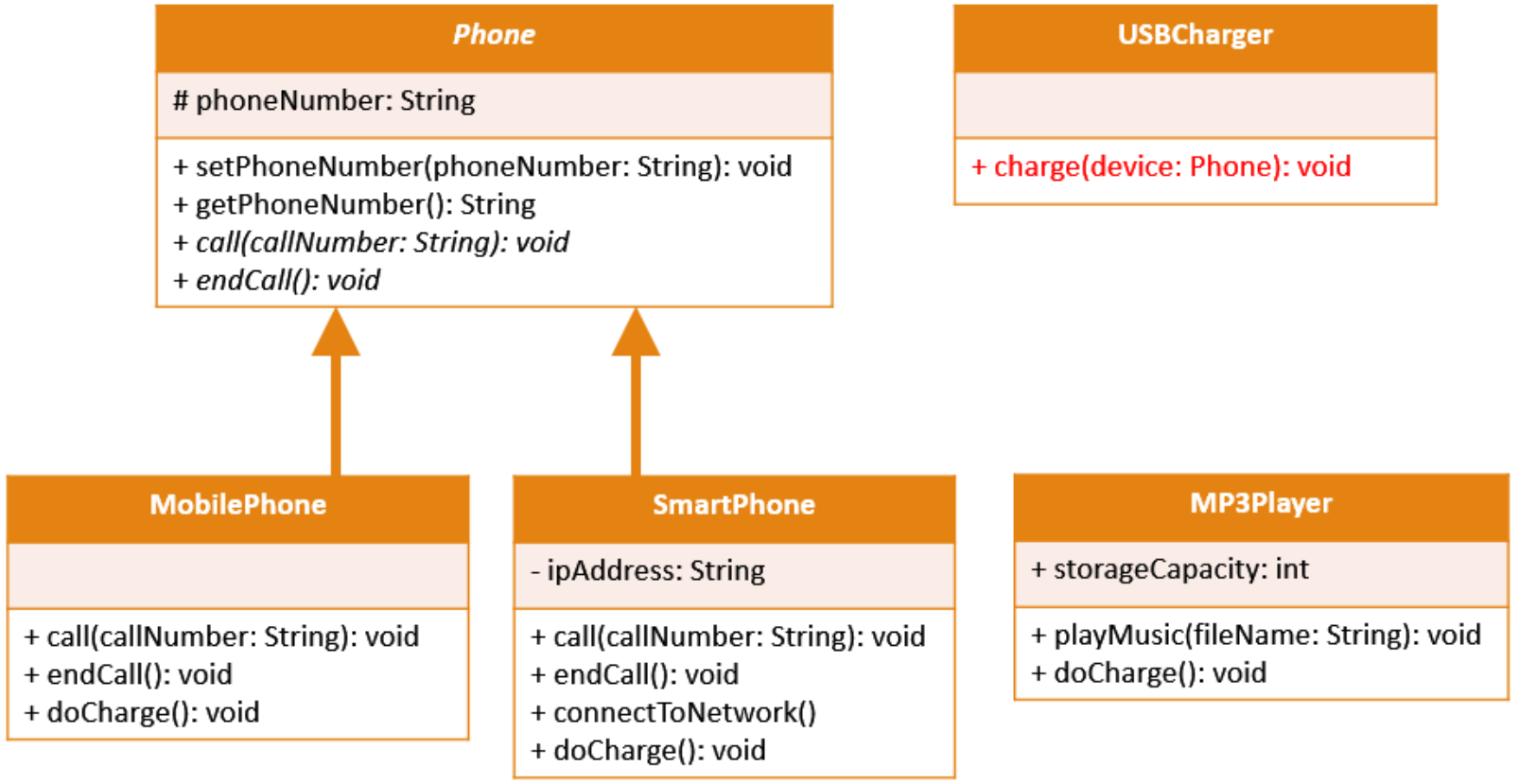
Example of an Abstract Class



Interface

- An interface is used as a bridge between objects so they can communicate with each other without seeing the inheritance hierarchy of those objects.

Case Example



Let's think about it:

- In the class diagram above, we can see that the class MP3Player has no hierarchical relationship with the classes Phone, MobilePhone, or SmartPhone.
- As a result, the class MP3Player cannot be passed as a parameter in the method charge() that exists in the class USBCharger.
- Meanwhile, the classes MobilePhone and SmartPhone can be passed to the method charge() because they are descendants of the class Phone.
- The hope is that all classes can be passed to the method charge().
- What is the solution?
- We can use an interface.

How to Create an Interface

- To create an interface, use the keyword interface:

```
3    public interface ChargeAble
4    {
5
6    }
```

Methods in an Interface

- Unlike an abstract class, an interface cannot have methods with implementations.
- Methods inside an interface are only declarations (ended with a semicolon, without any curly brackets).

```
3    public interface ChargeAble
4    {
5        public void doCharge();
6    }
```

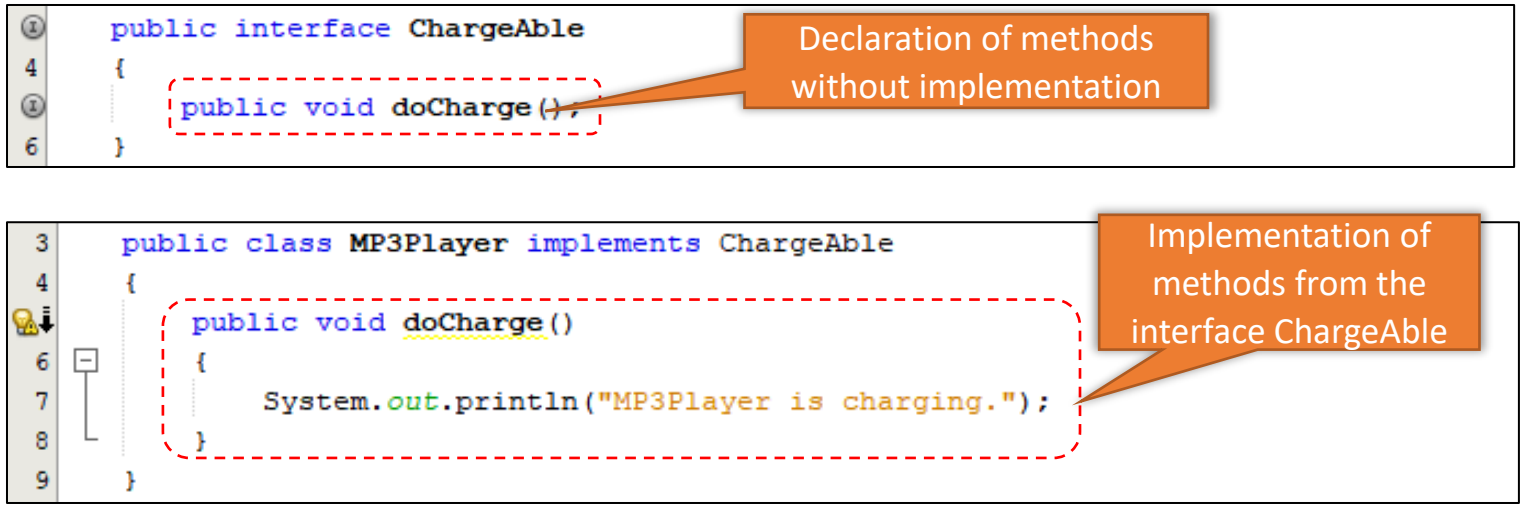
Classes that Implement an Interface

- An interface can be implemented by other classes. The way is by using the keyword implements.

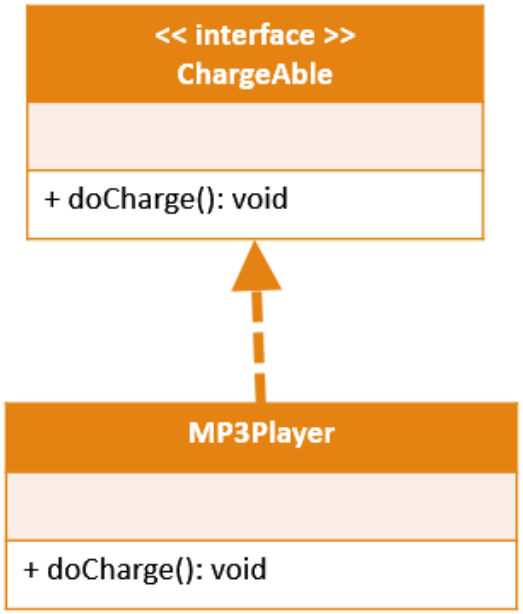
```
3    public class MP3Player implements ChargeAble
4    {
5
6    }
```

Classes that Implement an Interface

- All methods in the interface must be reimplemented in the class that implements it.



Interface notation in UML Class Diagram



- In the interface, an << interface >> s e.
- Then the class that implements the interface is marked with an arrow and a dashed line pointing to the interface.

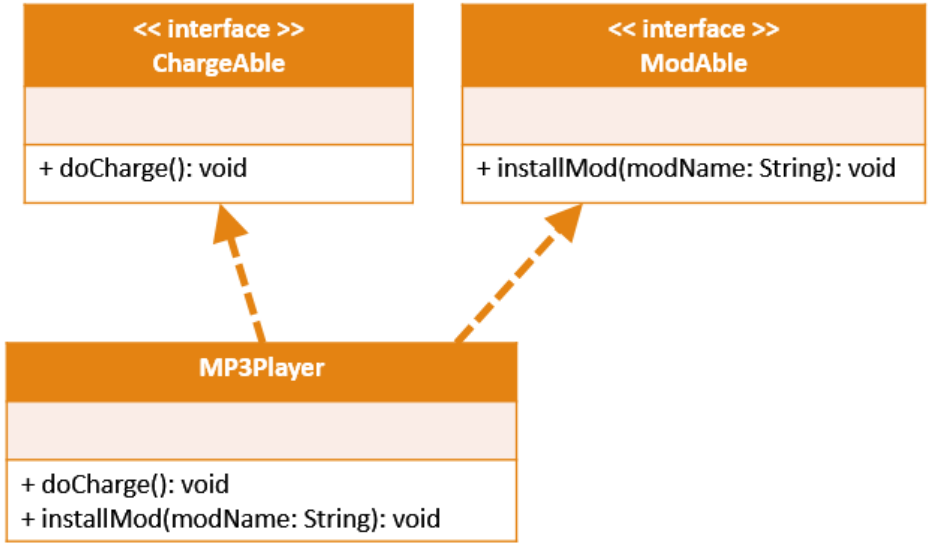
Multiple Interface

- The advantage of the interface is that one class can implement more than one interface.
- Therefore, the interface is said to be a solution if we want to apply multiple inheritance.
- Writing implements of more than one interface is separated by commas (,)

```
3    public class MP3Player implements ChargeAble, ModAble
4    {
5        // ...
6    }
```

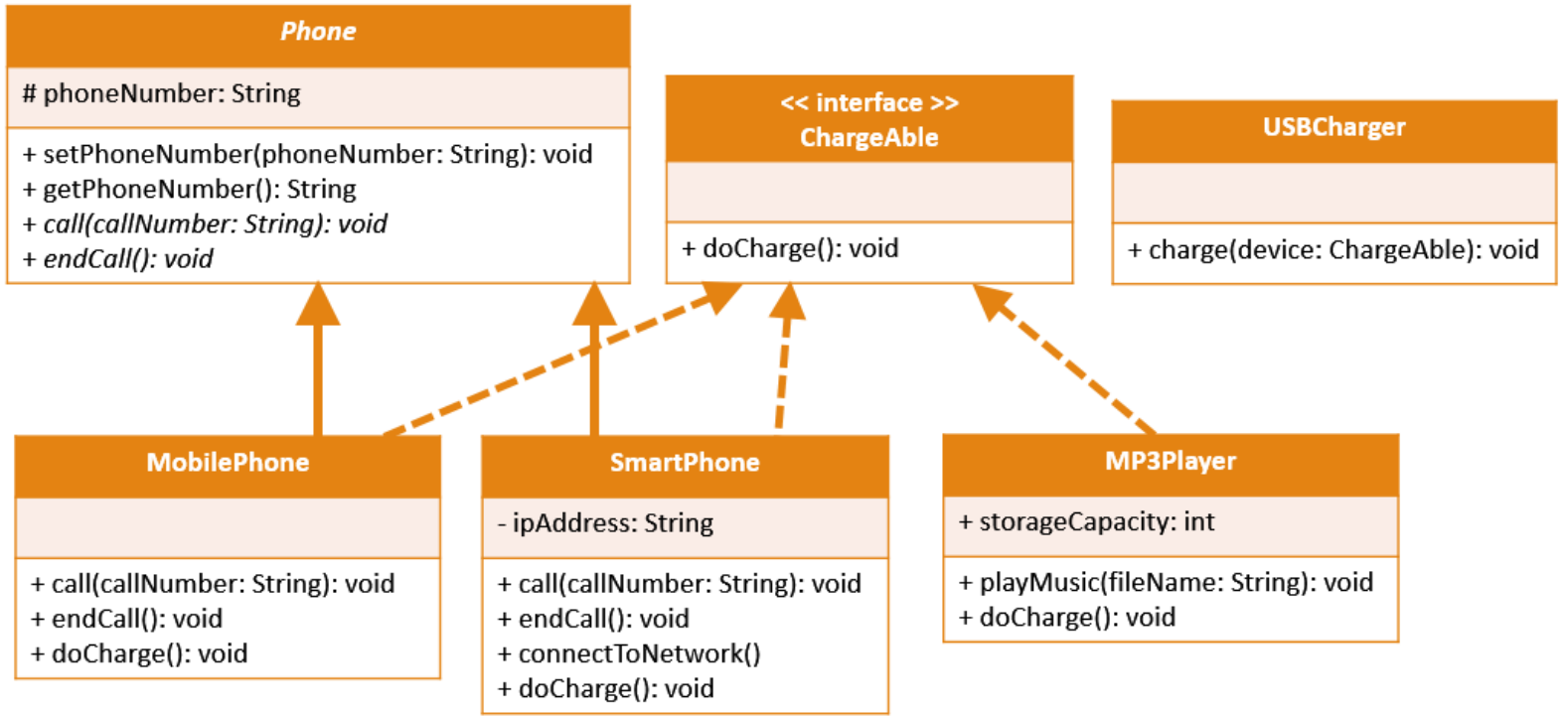
Example of Multiple Interface

- For example, the MP3Player class that implements ChargeAble and ModAble:



- Thus, the MP3Player class must implement all methods that exist in ChargeAble and ModAble.

Applying Interface on Case Example



Nice Solution!

- We can observe in the class diagram above, the classes MobilePhone, SmartPhone, and MP3Player all implement the ChargeAble interface.
- These classes also re-implement the method doCharge().
- Then in the USBCharger class, the charge() method now accepts a parameter of type ChargeAble
- **Thus, all classes that implement ChargeAble can be charged by USBCharger.**

When to Use Abstract Class or Interface?

- Use an abstract class if:
  - We want to share code/inherit the same code with related classes.
  - We want to create several classes with similar methods and properties.
  - We want to declare properties that are neither static nor final. This way, we can define methods with different implementations in each class that inherits the abstract class.
- Use an interface if:
  - We want to implement an interface in several classes that are not hierarchically related.
  - We want to specify the behavior of several objects, but do not want to know how the implementation of that behavior works.
  - We want to take advantage of multiple inheritance.

Question?